Far-Field Transform via Vector Cylindrical Harmonics: Reformulation for Large Oblique Angles

FFTLog Hankel Transforms and
Graf's Addition Theorem for Local-Basis Shift

Julia Implementation

Based on notes by Arvin Keshvari (Supervisor: Dr. Zin Lin)

March 26, 2026

Contents

Overview

The original algorithm computes far-field coefficients $A_m(k_r)$ in a cylindrical basis centered at the optical axis and then reconstructs the field at the off-axis focal spot $(x_0, 0) = (f \tan\alpha, 0)$. For large oblique angles this requires both (i) high-order ($|m| \sim M_{\max} = k \sin\alpha R$) Hankel transforms and (ii) evaluating a slowly-converging cylindrical-harmonic sum at large $r = x_0$.

This note describes two replacements:

1. **FFTLog** for all Hankel transforms — order-agnostic, stable at $|m| \sim 600$.
2. **Graf's addition theorem** to shift the cylindrical basis from the optical axis to the focal spot entirely in modal (spectral) space — no Cartesian regridding, no interpolation, no 2D IFFT.

Steps 1–2: Angular Decomposition and Forward Hankel Transforms

These steps are identical to the previous reformulation and are retained without change.

Step 1. Decompose the near field via FFT over θ:

$$E^{\mathrm{N}}_{m,r/\theta}(r_j) = \frac{1}{N_\theta} \sum_l E_{r/\theta}(r_j, \theta_l)\, e^{-im\theta_l}, \qquad |m| \le M_{\max} = \lceil k \sin\alpha R \rceil + \delta M.$$

Step 2. Compute TE and TM coefficients via FFTLog (4 calls per mode):

$$A^{\mathrm{TE}}_m(k_r) = \tfrac{i}{2}\big[\mathcal{H}_{m+1}[rE^{\mathrm{N}}_{m,r}] + \mathcal{H}_{m-1}[rE^{\mathrm{N}}_{m,r}]\big] + \tfrac{1}{2}\big[\mathcal{H}_{m-1}[rE^{\mathrm{N}}_{m,\theta}] - \mathcal{H}_{m+1}[rE^{\mathrm{N}}_{m,\theta}]\big], \tag{1}$$

$$A^{\mathrm{TM}}_m(k_r) = -\tfrac{k_z/k}{2}\big[\mathcal{H}_{m-1}[rE^{\mathrm{N}}_{m,r}] - \mathcal{H}_{m+1}[rE^{\mathrm{N}}_{m,r}]\big] - i\tfrac{k_z/k}{2}\big[\mathcal{H}_{m-1}[rE^{\mathrm{N}}_{m,\theta}] + \mathcal{H}_{m+1}[rE^{\mathrm{N}}_{m,\theta}]\big]. \tag{2}$$

Output: $A_m(k_r)$ on a log-spaced k_r grid for $|m| \le M_{\max}$.

Step 3: Propagation to the Focal Plane

Instead of assembling the angular spectrum $\mathcal{U}(k_r, \phi)$ at this stage, we simply apply the propagation phase to each modal coefficient individually:

$$\tilde{A}_m(k_r) = A_m(k_r) \cdot e^{ik_z(k_r)\, f}, \qquad k_z(k_r) = \sqrt{k^2 - k_r^2}.$$

Evanescent modes ($k_r > k$, imaginary k_z) are set to zero. This is a pointwise multiplication at each (m, k_r) — no FFT, $O(N_{k_r} M_{\max})$.

The far field at $z = f$ in the *original* cylindrical frame is then

$$E(r, \theta, f) = \sum_m \left[\int_0^k \tilde{A}_m(k_r)\, J_m(k_r r)\, k_r\, dk_r \right] e^{im\theta}.$$

Step 4: Shifting the Basis via Graf's Addition Theorem

Graf's addition theorem

We wish to re-expand (**??**) in a cylindrical basis centered at the focal spot $\boldsymbol{x}_0 = (x_0, 0)$, where $x_0 = f \tan\alpha$. Define local polar coordinates (ρ, ψ) by

$$x = x_0 + \rho\cos\psi, \quad y = \rho\sin\psi.$$

Graf's addition theorem for Bessel functions [DLMF §10.23(ii)] states: for a shift $\boldsymbol{x}_0 = (x_0, 0)$ along the x-axis,

$$\boxed{J_n(k_r r)\, e^{in\theta} = \sum_{m=-\infty}^{\infty} J_m(k_r x_0)\, J_{n-m}(k_r \rho)\, e^{i(n-m)\psi}, \qquad \rho < x_0.}$$

The condition $\rho < x_0$ is the PSF regime: the local evaluation point is within a distance $x_0 = f\tan\alpha$ of the focal spot, which holds trivially for any reasonable PSF patch.

Convergence. The sum in (**??**) is absolutely convergent for $\rho < x_0$. The m-th term involves $J_m(k_r x_0)$, which decays exponentially for $|m| > k_r x_0$. The effective truncation of the outer sum is therefore at $|m| \lesssim k_r x_0 = k_r f \tan\alpha$. For the inner sum, only $|n - m| \lesssim k_r \rho_{\max}$ contribute significantly.

Local modal coefficients

Substituting (**??**) into the far-field sum and swapping the order of summation (let $l = n - m$, so $n = m + l$):

$$\begin{aligned} E(\rho, \psi, f) &= \sum_n \int_0^k \tilde{A}_n(k_r) \sum_m J_m(k_r x_0)\, J_{n-m}(k_r \rho)\, e^{i(n-m)\psi}\, k_r\, dk_r \\ &= \sum_l \left[\int_0^k \underbrace{\sum_m \tilde{A}_{m+l}(k_r)\, J_m(k_r x_0)}_{B_l(k_r)}\, J_l(k_r \rho)\, k_r\, dk_r \right] e^{il\psi}. \end{aligned} \tag{3}$$

The *local modal coefficients* $B_l(k_r)$ are defined by

$$\boxed{B_l(k_r) = \sum_{m=-M_{\max}}^{M_{\max}} \tilde{A}_{m+l}(k_r)\, J_m(k_r x_0).}$$

This is a **discrete convolution in mode index** m at each k_r, with the weight sequence $c_m(k_r) = J_m(k_r x_0)$ (which is fast-decaying for $|m| > k_r x_0$).

Why this is better than polar-to-Cartesian regridding. The previous approach assembled $\mathcal{U}(k_r, \phi)$ in polar k-space (via IFFT over m) and then interpolated it onto a Cartesian grid centered at $(k \sin\alpha, 0)$. Interpolation introduces errors that are hard to bound, and the polar-to-Cartesian mapping requires a 2D IFFT over a large grid. The Graf approach is spectrally exact: (**??**) is an algebraic identity up to mode truncation. The truncation error is exponentially small for $\rho_{\max} < x_0$, and the outer sum (**??**) requires only $|l| \le L_{\max} \ll M_{\max}$ terms (Section **??**).

Step 5: Inverse Hankel Transforms in the Local Basis

Equation (**??**) shows that in the local basis the field is simply a sum of cylindrical harmonics, each obtained by an inverse Hankel transform of B_l:

$$b_l(\rho) = \int_0^\infty B_l(k_r)\, J_l(k_r \rho)\, k_r\, dk_r = \mathcal{H}_l^{-1}[B_l\, k_r](\rho).$$

Since the Hankel transform is self-inverse, this is another FFTLog call of order l:

$$b_l(\rho) = \mathrm{FFTLog}_l\big[k_r B_l(k_r)\big](\rho).$$

Local mode truncation

The local field is needed only within a PSF patch of radius $\rho_{\max}$ (a few Airy radii). The l-th local mode is significant only when $k_r \rho_{\max} \gtrsim |l|$, so the effective truncation is

$$L_{\max} \approx k \cdot \mathrm{NA} \cdot \rho_{\max}.$$

For $\mathrm{NA} = 0.5$, $\rho_{\max} = 5\lambda$: $L_{\max} \approx \pi \cdot 5 \approx 16$. Compare $M_{\max} \approx 600$ for the original basis. **The inverse Hankel transforms are both low-order and few in number** — this step is cheap regardless of α.

Step 6: Angular Synthesis in the Local Basis

The PSF in local polar coordinates is reconstructed from the local mode amplitudes by IFFT over l:

$$E(\rho_j, \psi_s, f) = \sum_{l=-L_{\max}}^{L_{\max}} b_l(\rho_j)\, e^{il\psi_s}.$$

This is an IFFT over l at each radial output point ρ_j, costing $O(N_\rho L_{\max} \log L_{\max})$ — negligible.

The result is the complex field amplitude in the local cylindrical frame centered at $(x_0, 0) = (f \tan\alpha, 0)$. Take $|E|^2$ for the PSF intensity.

Complete Improved Algorithm

Cy-FFP with Graf Shift — Julia

Inputs: E_r, E_θ on log-spaced r, uniform θ; k, α, f.

Step 1 — Angular FFT (over θ): $E^{\mathrm{N}}_{m,r/\theta}(r_j)$. Truncate at $M_{\max} = \lceil k \sin\alpha R \rceil + \delta M$.

Step 2 — Forward Hankel transforms (FFTLog, orders $m \pm 1$): $A^{\mathrm{TE}}_m(k_r)$, $A^{\mathrm{TM}}_m(k_r)$ for $|m| \le M_{\max}$.

Step 3 — Propagation: $\tilde{A}_m(k_r) = [A^{\mathrm{TE}}_m(k_r) + A^{\mathrm{TM}}_m(k_r)] \cdot e^{ik_z f}$, zero for $k_r > k$.

Step 4 — Graf shift (mode convolution at each k_r):

$$B_l(k_r) = \sum_{|m| \le M_{\max}} \tilde{A}_{m+l}(k_r)\, J_m(k_r x_0), \quad |l| \le L_{\max}, \quad x_0 = f \tan\alpha.$$

Step 5 — Inverse Hankel transforms (FFTLog, order l): $b_l(\rho_j) = \mathcal{H}_l^{-1}[k_r B_l(k_r)](\rho_j)$ for $|l| \le L_{\max}$.

Step 6 — Angular IFFT (over l): $E(\rho_j, \psi_s) = \sum_l b_l(\rho_j) e^{il\psi_s}$.

Comparison of Approaches

	Previous (polar → Cartesian)	This work (Graf shift)
Steps 3–6	IFFT over m; interpolate; 2D IFFT	Propagation; mode convolution; FFTLog$_l$; IFFT over l
Interpolation	Bilinear (accuracy-limited)	None — spectrally exact
Output grid	Cartesian (ξ, η)	Polar (ρ, ψ) local
PSF grid size	Fixed Cartesian patch	Fixed polar patch
Modes needed (output)	$\sim (k\,\mathrm{NA}\,\rho_{\max})^2$ Cartesian pts	$L_{\max} \sim k\,\mathrm{NA}\,\rho_{\max}$
Inverse HT orders	None	$\lvert l\rvert \le L_{\max} \ll M_{\max}$
Mode convolution cost	None	$O(N_{k_r} L_{\max} M_{\max})$
Accuracy vs α	Degrades (larger grid, sparser polar)	Uniform (exponential convergence)

Cost of the mode convolution (Step 4)

At each k_r, $B_l = \sum_m \tilde{A}_{m+l} J_m(k_r x_0)$ is a convolution of length $2M_{\max} + 1$ with a Bessel-function weight sequence. Two options:

1. **Direct** for small $L_{\max}$: $O(L_{\max} M_{\max})$ per k_r. For $L_{\max} = 16$, $M_{\max} = 600$: $\sim 10^4$ ops per k_r.

2. **FFT-based convolution**: zero-pad to length $2M_{\max}$, FFT, multiply by $\widehat{J}(k_r x_0)$, IFFT. $O(M_{\max} \log M_{\max})$ per k_r regardless of $L_{\max}$.

For typical parameters ($N_{k_r} \sim 512$, $M_{\max} \sim 600$, $L_{\max} \sim 20$), the direct approach costs $\sim 3 \times 10^6$ flops — negligible.

When $x_0 \to 0$ (paraxial / normal incidence)

For $\alpha \to 0$, $x_0 \to 0$ and $J_m(k_r x_0) \to \delta_{m,0}$, so $B_l \to \tilde{A}_l$. The Graf shift reduces to the identity: no change of basis is needed, which is the correct paraxial limit.

Julia Implementation Notes

The full implementation is in `CyFFP.jl`. The new functions replacing Steps 3–6 are:

Listing 1: Step 4: Graf mode convolution

```julia
"""
    graf_shift(A_tilde, kr_grid, x0, L_max) -> B[Nkr, 2*L_max+1]

Apply Graf's addition theorem in mode space.
At each kr, compute:
    B_l(kr) = sum_m A_tilde[m+l](kr) * J_m(kr * x0)
for |l| <= L_max.

The weight sequence J_m(kr*x0) decays for |m| > kr*x0,
so the effective sum length is min(M_max, ceil(kr*x0) + buffer).
"""
function graf_shift(A_tilde::Vector{ComplexF64},   # [2M_max+1] at one kr
                    kr_x0::Float64,                  # kr * x0
                    m_list::Vector{Int},
                    L_max::Int)
    M_max  = maximum(abs.(m_list))
    l_list = collect(-L_max:L_max)
    Nl     = length(l_list)
    B      = zeros(ComplexF64, Nl)

    # Bessel weights J_m(kr*x0): nonzero only for |m| < kr*x0 + buffer
    m_cut  = min(M_max, ceil(Int, kr_x0) + 20)
    ms     = -m_cut:m_cut

    # Evaluate J_m(kr*x0) for needed m values
    Jm_vals = Dict(m => besselj(m, kr_x0) for m in ms)

    for (li, l) in enumerate(l_list)
        acc = zero(ComplexF64)
        for m in ms
            n = m + l               # index into A_tilde
            if abs(n) <= M_max
                n_idx = n + M_max + 1   # 1-based index
                acc  += A_tilde[n_idx] * Jm_vals[m]
            end
        end
```

```
        B[li] = acc
    end
    return B
end
```

Listing 2: Step 5: Inverse Hankel transforms in local basis

```
"""
    local_hankel_inverse(B, kr_grid, dln, L_max) -> b[Nr_out, 2*L_max+1]

For each local mode l, compute the inverse Hankel transform:
    b_l(rho) = H_l^{-1}[kr * B_l(kr)](rho)
via FFTLog at order l.  Since H_nu is self-inverse, this is another
fftlog_hankel call with the same dln.
"""
function local_hankel_inverse(B::Matrix{ComplexF64},  # [Nkr, 2L_max+1]
                              kr_grid::Vector{Float64},
                              dln::Float64,
                              L_max::Int)
    Nkr  = size(B, 1)
    Nl   = 2*L_max + 1
    b    = zeros(ComplexF64, Nkr, Nl)
    l_list = collect(-L_max:L_max)

    for (li, l) in enumerate(l_list)
        # Integrand: kr * B_l(kr) (the kr factor comes from the Hankel
            measure)
        integrand = kr_grid .* B[:, li]
        # FFTLog inverse Hankel transform at order l
        b[:, li] = fftlog_hankel(integrand, dln, Float64(l))
    end
    return b
end
```

Listing 3: Step 6: Angular synthesis to get local PSF

```
"""
    synthesize_local_psf(b, rho_grid, Npsi) -> (psf[Nr, Npsi], psi_grid)

Given local cylindrical-harmonic amplitudes b[Nr, 2*L_max+1],
reconstruct the PSF via IFFT over l:
    E(rho_j, psi_s) = sum_l b_l(rho_j) * exp(i*l*psi_s)
"""
function synthesize_local_psf(b::Matrix{ComplexF64},
                              L_max::Int,
                              Npsi::Int)
    Nr    = size(b, 1)
    l_list = collect(-L_max:L_max)
    psi   = range(0.0, 2pi, length=Npsi+1)[1:end-1]
    psf   = zeros(ComplexF64, Nr, Npsi)

    for j in 1:Nr
        buf = zeros(ComplexF64, Npsi)
        for (li, l) in enumerate(l_list)
```

```
            jj      = mod(l, Npsi) + 1    # FFTW mode index
            buf[jj] += b[j, li]
        end
        psf[j, :] = ifft(buf) .* Npsi
    end
    return psf, collect(psi)
end
```